

Tabiquero, Khristopher Reeve V.
CCS 303-CS31S1 - Intelligent Agents

1. A video recording of your final output (the simple prototype - record from the start to end)

https://drive.google.com/drive/folders/1SIOZbSJK9fC8frh6pY0SsrwgKmgVWIMK?usp=s_haring

2. The python file

https://drive.google.com/drive/folders/1GyVvKCyG4u82o01hsHcf3yF0aR7fneMZ?usp=s_haring

3. A simple documentation which includes the following - a simple step-by-step how you did your final project. This should start from the dataset, annotation, training, testing and prototyping.

Rock-Paper-Scissors Gesture Recognition Project

Project Overview

The aim of this project is to develop a fun and interactive rock-paper-scissors game that uses hand gestures. By using a webcam, the program recognizes hand gestures in real-time and compares them against computer-generated moves to decide the winner. This process involves collecting data, training a machine learning model, and integrating it with gesture recognition.

1. Steps to Complete the Project

1. Data Collection

- Captured hand gesture data in real-time using a webcam.
- Saved this data into a file named `hand_gestures_CASTILLO.csv`.
- The file stores information about finger joint positions (X, Y, Z coordinates) and the associated gesture (e.g., "Rock", "Paper", "Scissors").

2. Data Annotation

- The data is automatically labeled (annotated) during collection.
- Each entry in the dataset includes hand movements and the recognized gesture.

3. Model Training

- Used a machine learning algorithm to create a model capable of recognizing gestures.
- Steps involved:
 - Loading the dataset from the CSV file.

- Splitting the data into training and testing groups.
- Training the model with gesture data.
- Evaluating its accuracy.

4. Real-Time Gesture Recognition

- Integrated the trained model with a webcam to predict gestures in real time.
- Matched the player's gesture against the computer's move to determine the winner.

2. Libraries and Tools Used

- **MediaPipe:** For detecting and tracking hand movements.
- **Pandas:** To handle and manipulate the dataset.
- **NumPy:** For calculations on numeric data.
- **Matplotlib:** For visualizing the decision tree.
- **Scikit-Learn:**
 - Trained the model using a Random Forest Classifier.
 - Evaluated performance using metrics like confusion matrix, accuracy score, and classification report.
- **Joblib:** Saved the trained model for future use.

3. Functions Overview

- **Data Preparation Functions:**
 - `importdata`: Loads the gesture dataset from the CSV file.
 - `splitdataset`: Divides the dataset into training and testing sets.
- **Training Functions:**
 - `train_using_random_forest`: Trains a Random Forest Classifier using gesture data.
- **Evaluation Functions:**
 - `prediction`: Uses the model to predict gestures in the test set.
 - `cal_accuracy`: Measures how well the model performs.
- **Visualization:**
 - `visualize_decision_tree`: Creates a graphical representation of the decision tree from the model.